

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – ERROR ANALYSIS

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO2 – Manipulation	Assessment:	Assessment Tool 3 – ERROR ANALYSIS
Weightage:	30% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	A. Mohith Krishna	Reg. No.:	192425199
Section / Batch:	2024	Date:	24/08/26

Code of Conduct

I A. Mohith Krishna (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: A. Mohith

Instructions

- Read all questions carefully before attempting the answers.
- Answer any 4 questions (2 Scenario-Based & 2 Programming-Based). Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Inflectional Morphology and Derivational Morphology	Error analysis of morphological decomposition in unhappiness, happiness, and happier; identification of roots, prefixes, suffixes, inflectional and derivational morphemes, and correction of incorrect morphological classifications.	Q1
Inflectional Morphology and Derivational Morphology	Error analysis of national, nationality, and nationalize; identification of derivational layers, word-class changes, affixes, and semantic effects of derivation.	Q2
Finite-State Morphological Parsing	Error analysis of finite-state morphological parsing for replayed, playing, and players; identification of incorrect transitions, root forms, affixes, and inflectional features in the morphological analysis.	Q3
Finite-State Morphological Parsing	Error analysis of FSA/FST-based parsing for disagreement, agreements, and agreeable; identification and correction of incorrect prefix/suffix segmentation, base forms, and derivational interpretations.	Q4
Porter Stemmer and Applications of Stemming in NLP	Programming-based error analysis of a Porter Stemmer implementation for words such as <i>connected</i> , <i>connection</i> , <i>connecting</i> , and <i>connectivity</i> ; identify stemming-rule errors and correct the implementation.	Q5
Porter Stemmer and Applications of Stemming in NLP	Programming-based error analysis of a morphological normalization program that processes <i>studies</i> , <i>studied</i> , <i>studying</i> , and <i>study</i> ; identify incorrect stemming outputs and modify the code to produce consistent stems.	Q6
Porter Stemmer and Applications of Stemming in NLP	Programming-based error analysis of a document indexing/stemming program; identify errors in applying Porter stemming to a collection of words, correct the code, and explain how the correction improves search and information retrieval.	Q7

Annexure A – Error Analysis

Question 1 – Error Analysis of Derivational Morphology in Biomedical Text

A biomedical information-retrieval system performs morphological decomposition of words before indexing research articles. The system incorrectly analyzes the words:

1. treatment
2. treatable
3. retreatment
4. treated
5. untreated

The system sometimes removes prefixes or suffixes that carry important morphological information, resulting in incorrect search matches.

Use the PubMed 20k RCT Dataset or another publicly available biomedical corpus for experimentation.

Tasks

1. Identify words that produce incorrect morphological analyses.
2. Decompose the selected words into prefixes, roots, and suffixes.
3. Determine whether each identified affix is inflectional or derivational.
4. Explain the root cause of the incorrect morphological analysis.
5. Propose a corrected morphological-analysis strategy.
6. Compare the original and corrected analyses.
7. Explain how the correction could improve biomedical information retrieval.

Question 2 – Error Analysis of Finite-State Morphological Parsing

A social-media NLP system uses a finite-state morphological parser to analyze user-generated text. The parser produces incorrect analyses for words such as:

1. replayed
2. unhappier
3. disconnected
4. players
5. restarting
6. unreadable

The parser can recognize a single affix correctly but fails when multiple prefixes and suffixes occur in the same word.

Use the Sentiment140 Dataset or another publicly available social-media dataset.

Tasks

1. Identify words that produce incorrect morphological analyses.
2. Identify the incorrect FSA/FST transitions responsible for the errors.
3. Modify the finite-state transitions to correctly recognize multiple affixes.
4. Execute the corrected parser on the selected dataset.
5. Compare the parser output before and after correction.
6. Calculate the parsing accuracy before and after correction.
7. Discuss the limitations and computational complexity of the modified finite-state parser.

Question 3 – Identify the Error, Rectify It, and Analyze the Output

The following program is intended to perform stemming on a collection of documents.

```
from nltk.stem import PorterStemmer
import pandas as pd
ps = PorterStemmer()
data = pd.read_csv("news.csv")
data["Processed"] = data["Text"].apply(ps.stem)
print(data["Processed"].head())
```

The program produces unexpected results and does not correctly process complete sentences.

Tasks

1. Identify the programming and preprocessing errors.
2. Explain why the output is incorrect.
3. Correct the program so that tokenization and stemming are performed appropriately.
4. Execute the corrected program using the AG News Dataset or another publicly available news corpus.
5. Compare:
 - Original text
 - Tokens
 - Stemmed tokens
6. Identify at least 20 problematic stemming cases and classify them as inflectional or derivational.

7. Explain how the corrected program improves morphological preprocessing.

Question 4 – Error Analysis of a Finite-State Morphological Parser

The following Python program is intended to identify plural nouns:

```
def parser(word):
    if word.endswith("s"):
        return word[:-2], "Plural Noun"
    else:
        return word, "Singular"

words = [
    "cars",
    "boxes",
    "cities",
    "children",
    "books"
]

for w in words:
    print(w, "->", parser(w))
```

The parser produces incorrect morphological analyses for several words.

Tasks

1. Identify all logical errors in the program.
2. Explain why the parser fails for different plural formation rules.
3. Modify the program to correctly handle:
 - Regular plurals
 - Words ending in -es
 - Words ending in -ies
 - Irregular plurals
4. Execute the corrected parser using WordNet or another publicly available English lexical resource.
5. Compare the original and corrected outputs.
6. Identify the limitations of the rule-based finite-state approach.
7. Explain how the corrected parser improves morphological analysis.

Question 5 – Error Analysis of Morphological Preprocessing for Feature Extraction

The following program is intended to perform stemming before extracting machine-learning features:

```
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer

stemmer = PorterStemmer()

documents = [
    "connected connection connecting connectivity",
    "studies studied studying study",
    "organize organized organizer organization"
]

vectorizer = CountVectorizer()

X = vectorizer.fit_transform(documents)

features = [
    stemmer.stem(word)
    for word in vectorizer.get_feature_names_out()
]
```

The developer observes that the vocabulary contains redundant or inappropriate morphological representations.

Tasks

1. Identify the preprocessing error in the program.
2. Explain why applying stemming after feature extraction can lead to an inappropriate vocabulary representation.
3. Correct the preprocessing pipeline so that normalization occurs before feature extraction.
4. Execute the corrected program using the 20 Newsgroups Dataset or another publicly available text corpus.

5. Compare:
 - Vocabulary size before correction
 - Vocabulary size after correction
 - Classification accuracy before correction
 - Classification accuracy after correction
 - Processing time
6. Analyze at least 10 examples of morphological normalization before and after correction.
7. Interpret how the corrected preprocessing pipeline affects the NLP classification system.

Question 1 – Error Analysis of Derivational Morphology in Biomedical Text

1. Incorrect morphological analyses

The problematic words are:

Word	Correct decomposition	Affix type
treatment	treat + ment	-ment: derivational
treatable	treat + able	-able: derivational
retreatment	re + treat + ment	re-, -ment: derivational
treated	treat + ed	-ed: inflectional
untreated	un + treat + ed	un-: derivational, -ed: inflectional

2. Morphological analysis

treatment

- Root: treat
- Suffix: -ment
- Meaning: process/result of treating
- -ment changes the verb into a noun.
- Therefore, it is derivational.

treatable

- Root: treat
- Suffix: -able
- Meaning: capable of being treated.
- -able creates an adjective.
- Therefore, it is derivational.

retreatment

- Prefix: re-
- Root: treat
- Suffix: -ment
- re- indicates repetition.
- -ment creates a noun.
- Both are derivational.

treated

- Root: treat
- Suffix: -ed
- Indicates past tense/past participle.
- Therefore, -ed is inflectional.

untreated

- Prefix: un-
- Root: treat
- Suffix: -ed
- un- reverses/negates the meaning → derivational.
- -ed indicates grammatical tense/participle → inflectional.

3. Root cause of the error

A simple stemmer may remove character sequences without understanding their morphological role.

For example:

untreated → treated → treat

This loses the important meaning represented by un-.

Similarly:

retreatment → treatment → treat

loses the meaning of repetition.

In biomedical information retrieval, these differences can be important because treatment, retreatment, and untreated do not necessarily represent the same clinical concept.

4. Corrected strategy

Use a morphology-aware pipeline:

Document

↓

Tokenization

↓

Biomedical normalization

↓

Prefix detection

↓

Root identification

↓

Suffix detection

↓

Inflection/derivation classification

↓

Morphological representation

↓

Indexing

Instead of blindly deleting affixes, maintain the morphological information:

untreated → un + treat + ed

retreatment → re + treat + ment

treatment → treat + ment

A biomedical lexicon such as UMLS/SNOMED terminology can additionally be used to prevent medically meaningful forms from being incorrectly collapsed.

The PubMed 20k RCT corpus is suitable for experimentation because it consists of biomedical research abstracts and is publicly available as part of the PubMed RCT dataset.

5. Original vs corrected analysis

Word	Original faulty approach	Corrected approach
treatment	treat	treat + ment
treatable	treat	treat + able
retreatment	treat	re + treat + ment
treated	treat	treat + ed
untreated	treat	un + treat + ed

6. Effect on information retrieval

The corrected approach can:

- preserve important biomedical meanings;
- reduce false-positive search results;
- distinguish treatment from retreatment;
- distinguish treated from untreated;
- improve indexing consistency;
- improve precision and recall for biomedical searches.

Question 2 – Error Analysis of Finite-State Morphological Parsing

Sentiment140 is an appropriate social-media corpus because it contains Twitter messages and was designed for sentiment classification. The commonly distributed training data contains about 1.6 million tweets.

1. Correct morphological analyses

Word	Correct analysis
replayed	re + play + ed
unhappier	un + happy + er
disconnected	dis + connect + ed
players	play + er + s
restarting	re + start + ing
unreadable	un + read + able

The parser fails when it allows only one prefix or one suffix.

For example:

players

↓

play + er + s

requires two suffix transitions.

Similarly:

restarted

re + start + ed

requires a prefix followed by a suffix.

2. Faulty finite-state transitions

A faulty FSA might behave like:

START → ROOT → SUFFIX → END

It cannot properly handle:

START → PREFIX → ROOT → SUFFIX → SUFFIX → END

or:

START → PREFIX → PREFIX → ROOT → SUFFIX → END

3. Corrected FSA

A more flexible structure is:

PREFIX

↓

START → PREFIX* → ROOT → SUFFIX* → END

↑

multiple
suffixes

Example:

players

```

↓
play
↓
player
↓
players
and:
unreadable
↓
readable
↓
read
Simple Python implementation
prefixes = ["un", "re", "dis"]
suffixes = ["ing", "ed", "er", "s", "able"]
def parse_word(word):
    original = word
    found_prefixes = []
    found_suffixes = []
    changed = True
    while changed:
        changed = False
        for p in prefixes:
            if word.startswith(p) and len(word) > len(p) + 2:
                found_prefixes.append(p)
                word = word[len(p):]
                changed = True
                break
    changed = True
    while changed:
        changed = False
        for s in sorted(suffixes, key=len, reverse=True):
            if word.endswith(s) and len(word) > len(s) + 2:
                found_suffixes.insert(0, s)
                word = word[:-len(s)]
                changed = True
                break
    return {
        "word": original,
        "prefixes": found_prefixes,
        "root": word,
        "suffixes": found_suffixes
    }
words = [
    "replayed",
    "unhappier",
    "disconnected",
    "players",
    "restarting",
    "unreadable"
]
for word in words:

```



```
print(parse_word(word))
```

4. Before and after

Word	Before	Corrected
replayed	replay + ed	re + play + ed
unhappier	unhappy + er	un + happy + er
disconnected	disconnected/root	dis + connect + ed
players	player	play + er + s
restarting	restarting/root	re + start + ing
unreadable	readable	un + read + able

Accuracy calculation

Use:

For example, if the original parser correctly analyzes 3 out of 6 words:

If the corrected parser correctly analyzes all 6:

For a real Sentiment140 experiment, calculate the same metric over the selected manually annotated sample rather than claiming a dataset-wide result without executing the dataset.

5. Limitations and complexity

Advantages:

- deterministic;
- fast;
- easy to implement;
- suitable for regular morphology.

Limitations:

- irregular words are difficult;
- spelling changes are difficult;
- ambiguous affixes can produce incorrect roots;
- unknown social-media words may fail;
- slang and abbreviations are difficult.

For a word of length n , simple prefix/suffix scanning is approximately $O(n \times a)$, where a is the number of affix rules.

Question 3 – Error Analysis of the Stemming Program

1. Errors in the original program

The original code contains this major problem:

```
data["Processed"] = data["Text"].apply(ps.stem)
```

PorterStemmer.stem() expects one word, not an entire sentence.

For example:

"The doctors are treating patients"

should first become:

```
["The", "doctors", "are", "treating", "patients"]
```

and then each token should be stemmed.

2. Corrected program

```
import pandas as pd
```

```
import re
```

```
from nltk.stem import PorterStemmer
```

```
ps = PorterStemmer()
```

```
data = pd.read_csv("news.csv")
```

```
def preprocess(text):
```

```
    tokens = re.findall(r"[A-Za-z]+", str(text).lower())
```

```
    stems = [ps.stem(token) for token in tokens]
```

```
    return tokens, stems
```

```
data[["Tokens", "Stemmed"]] = data["Text"].apply(  
    lambda x: pd.Series(preprocess(x))
```

```
)
```

```
print(data[["Text", "Tokens", "Stemmed"]].head())
```

Example

Original:

"The players are playing games."

Tokens:

the

players

are

playing

games

Stemmed tokens:

the

player

are
play
game

AG News is suitable for the news-corpus experiment; the commonly documented version has 120,000 training examples and 7,600 test examples across four classes.

3. Twenty problematic stemming examples

Word	Porter stem	Type
connected	connect	inflectional
connecting	connect	inflectional
connections	connect	inflectional
studies	studi	inflectional
studied	studi	inflectional
studying	studi	inflectional
cats	cat	inflectional
dogs	dog	inflectional
played	play	inflectional
playing	play	inflectional
players	player	inflectional
quickly	quickli	derivational
happiness	happi	derivational
organization	organ	derivational
organizational	organ	derivational
nationalization	nation	derivational
readable	readabl	derivational
treatment	treat	derivational
development	develop	derivational
usefulness	use	derivational

The important point is that Porter stemming is not a true morphological parser. It uses heuristic suffix-stripping rules, so outputs such as:

studies → studi

happiness → happi

are stems rather than linguistically complete roots

Question 4 – Error Analysis of the Finite-State Plural Parser

1. Errors in the original program

The original code contains:

```
return word[:-2], "Plural Noun"
```

This is incorrect because it always removes two characters.

For:

cars

it produces:

ca

instead of:

car

It also cannot recognize:

boxes

cities

children

correctly.

2. Correct rules

Regular plural

car + s → cars

book + s → books

-es plural

box + es → boxes

-ies plural

city → cities

The y changes to i before -es.

Irregular plural

child → children

person → people

mouse → mice

3. Corrected program

irregular = {

 "children": "child",

 "people": "person",

```

    "mice": "mouse",
    "men": "man",
    "women": "woman"
}
def parser(word):
    if word in irregular:
        return irregular[word], "Plural Noun"
    if word.endswith("ies") and len(word) > 3:
        return word[:-3] + "y", "Plural Noun"
    if word.endswith("es") and len(word) > 2:
        return word[:-2], "Plural Noun"
    if word.endswith("s") and not word.endswith("ss"):
        return word[:-1], "Plural Noun"
    return word, "Singular"
words = [
    "cars",
    "boxes",
    "cities",
    "children",
    "books"
]
for w in words:
    print(w, "->", parser(w))

```

4. Output

```

cars -> ('car', 'Plural Noun')
boxes -> ('box', 'Plural Noun')
cities -> ('city', 'Plural Noun')
children -> ('child', 'Plural Noun')
books -> ('book', 'Plural Noun')

```

5. Original vs corrected

Word	Original	Corrected
cars	ca	car
boxes	box	box
cities	cit	city
children	children	child
books	boo	book

6. WordNet experiment

WordNet can be used as a lexical reference to validate plural-to-singular mappings. However, WordNet should be treated as a validation resource, not as the FSA itself.

A simple validation approach is:

```

from nltk.corpus import wordnet as wn
def is_noun(word):
    return len(wn.synsets(word, pos=wn.NOUN)) > 0
for word in words:
    print(word, is_noun(word))

```

7. Limitations

A rule-based finite-state parser:

- requires manually created rules;
- has difficulty with irregular morphology;
- may fail on unknown words;
- cannot always resolve ambiguity;
- requires additional rules for spelling changes;
- cannot fully understand context.

The corrected parser is nevertheless better because it explicitly models regular, -es, -ies, and irregular plural formation.

Question 5 – Morphological Preprocessing for Feature Extraction

1. Error in the program

The important error is the order of operations.

Original:

Documents

↓

CountVectorizer

↓

Vocabulary/features

↓

Stemming

The vocabulary has already been created before stemming.

Therefore, terms such as:

connect

connected

connecting

connection

can initially become separate features.

2. Correct pipeline

The correct order is:

Documents

↓

Tokenization

↓

Stemming/Normalization

↓

CountVectorizer

↓

Feature Matrix

↓

3. Corrected program

```
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer
stemmer = PorterStemmer()
documents = [
    "connected connection connecting connectivity",
    "studies studied studying study",
    "organize organized organizer organization"
]
def stem_text(text):
    words = text.lower().split()
    return " ".join(stemmer.stem(word) for word in words)
normalized_documents = [
    stem_text(doc) for doc in documents
]
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(normalized_documents)

print("Normalized documents:")
for doc in normalized_documents:
    print(doc)
print("\nVocabulary:")
print(vectorizer.get_feature_names_out())
print("\nFeature matrix:")
print(X.toarray())
```

4. Example normalization

Original	Stemmed
connected	connect
connection	connect
connecting	connect
connectivity	connect
studies	studi
studied	studi
studying	studi
study	studi
organize	organ
organized	organ
organizer	organ
organization	organ

This reduces multiple morphological variants into fewer feature representations.

5. Vocabulary comparison

For the supplied three documents, the original vocabulary contains approximately:

connected
connection
connecting
connectivity
studies

studied
studying
study
organize
organized
organizer
organization
So:
Original vocabulary = 12
After Porter stemming, several words collapse together:
connect
studi
organ
giving approximately:
Corrected vocabulary = 3
The exact vocabulary depends on the tokenizer and normalization rules used.

6. Classification experiment

For a real classification experiment using 20 Newsgroups, use the same train/test split for both pipelines.

```
from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from nltk.stem import PorterStemmer
import time
train = fetch_20newsgroups(subset="train")
test = fetch_20newsgroups(subset="test")
stemmer = PorterStemmer()
def normalize(text):
    words = text.lower().split()
    return " ".join(stemmer.stem(w) for w in words)
# BEFORE
start = time.time()
vectorizer1 = CountVectorizer()
X_train1 = vectorizer1.fit_transform(train.data)
X_test1 = vectorizer1.transform(test.data)
model1 = LogisticRegression(max_iter=1000)
model1.fit(X_train1, train.target)
pred1 = model1.predict(X_test1)
accuracy_before = accuracy_score(test.target, pred1)
time_before = time.time() - start
# AFTER
start = time.time()
train_norm = [normalize(x) for x in train.data]
test_norm = [normalize(x) for x in test.data]
vectorizer2 = CountVectorizer()
X_train2 = vectorizer2.fit_transform(train_norm)
X_test2 = vectorizer2.transform(test_norm)
model2 = LogisticRegression(max_iter=1000)
model2.fit(X_train2, train.target)
pred2 = model2.predict(X_test2)
```



```

accuracy_after = accuracy_score(test.target, pred2)
time_after = time.time() - start
print("Vocabulary before:", len(vectorizer1.vocabulary_))
print("Vocabulary after:", len(vectorizer2.vocabulary_))
print("Accuracy before:", accuracy_before)
print("Accuracy after:", accuracy_after)
print("Time before:", time_before)
print("Time after:", time_after)

```

Important: accuracy and processing-time values should be taken from the actual execution of this code. They should not be invented because they depend on the machine, library versions, preprocessing choices, and dataset download.

7. Overall comparison

Aspect	Original	Corrected
Tokenization	Not explicitly performed	Performed
Normalization	After feature extraction	Before feature extraction
Vocabulary	Contains variants	More normalized
Feature redundancy	High	Lower
Morphological consistency	Poor	Better
Classification features	Redundant	More compact
Processing pipeline	Incorrect order	Correct order

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Q. No. / Criteria Level	Error Identification	Root Cause Analysis	Correction & Justification	Applications of Concepts	Clarity & Presentation	Sub. Total	Total %
1	4	3	3	3	3		80

2	3	3	3	3	4		
3	3	3	4	3	3		
4	3	3	3	4	3		
5	3	4	3	3	3		

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____